

MINIX

The operating system which Linux set out to emulate, uses a microkernel architecture. The kernel of MINIX is only around 4000 lines, compared to Linux's 13.5 million

From source

Sorcerer, SourceMage, Lunar Linux, and Gentoo are distros based around compiling all your applications from source code so they can be optimized for your hardware

hardware that is not fully supported. Such hardware may be supported in a newer version or through patches that can enable support for it.

The kernel is the most central part of the OS, and one which you cannot do without, but the only way to make your kernel run faster or use less memory is to re-compile it exactly the way you want.

Hardware support is a great place for optimising the kernel. When configuring how your kernel is built, you will be able to control which drivers are loaded as modules, and which are compiled into the kernel. If you have a hardware device that you do not use regularly, it is best to have it as a module.

For example, if your computer only has Linux installed, you may not need support for NTFS, however you can keep it around as a module in case someone brings over a portable drive formatted in NTFS.

Next we go to the actual steps of compiling your own Linux kernel.

Compiling your own kernel

Before you begin, we need to remind you. Compiling the kernel is a long, CPU-intensive process, that requires large amounts of disk space.

Now many Linux distributions have their own version of the kernel, with their own customisations, and often the kernel build procedure can change because of

this. So, instead of using your own distribution's kernel source code we will ask you to download the latest untouched "vanilla" kernel source code from the <http://www.kernel.org> web site, and build that instead. Before we go ahead, download the latest kernel sources and place them in the "/usr/src" directory.

```
cd /usr/src
sudo tar xvf /path/to/download/sources/linux-2.6.36.tar.bz2
```

At the time of writing this article, the latest kernel version was 2.6.36, hence we will use "/usr/src/linux-2.6.36/" as the directory containing the sources. Replace this with the directory where you extracted your sources.

STEP 1: Getting the build requirements ready

Building the Linux kernel requires quite a few packages to be installed. The usual build tools such as GCC and make are required, but the kernel has a few more dependencies for successfully compiling. A list of the tools you need will be listed in the "Documentation/Changes" file included with the kernel sources.

Whichever distribution you use will probably include build tools for the kernel available as a package / meta-package / pattern etc. Simply install that and you should be good to go. Some features of the build process however, might have

additional dependencies. For example, if you plan to use the graphical configuration tools, you need to have GTK or QT libraries installed. If you try to use these commands without the requisite packages installed they will output exactly what you need to install.

To keep everything neat and clean, we will use a separate directory for our compilation output. In our case we have created a directory called "linux" under our home folder, but you could place this anywhere.

It is a good idea to copy out the current kernel's configuration to the one you are building to serve as a starting point, so you don't need to configure

everything. To do this, enter the directory where you want to build your kernel (in our case "~/linux" or "/home/username/linux"), and type the following:

```
zcat /proc/config.gz > .config
```

Most distribution kernels give access to the configuration via "/proc/config.gz", with this command we are extracting that information into a file called ".config" in the build directory. In case your distribution does not have "/proc/config.gz", you could look in "/boot" for the config file, which will likely be named "config-" followed by your kernel version. Copy this file to the build directory as ".config". If neither of these exist, you should check with your distribution's documentation for more information about how to locate this file, or simply start from scratch.

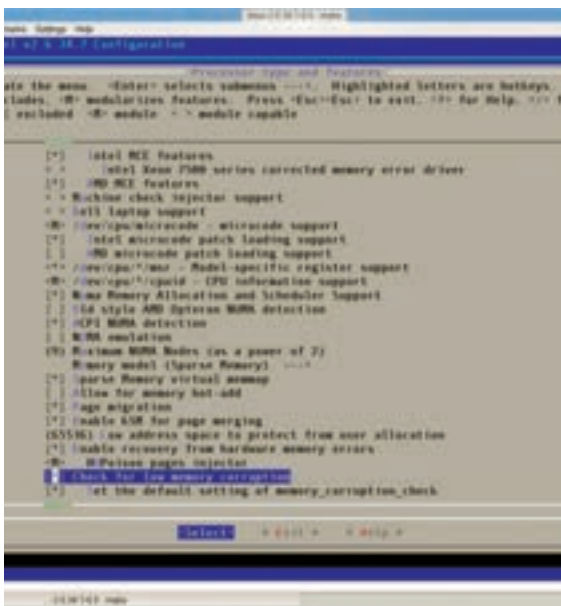
STEP 2: Configuring the kernel

This section alone can have a few thousand pages dedicated to it! We will instead give you just a singular piece of advice: If you don't understand what a setting does, don't touch it.

There are many ways to configure the kernel. The first is "wizard"-like method "make config" this will walk you through each one of the thousands of kernel configuration parameters one-by-one, we do not recommend you use this option. The second similar option is "make oldconfig", which you can use if you already copied the config of your running kernel in the last step. This will only ask you to configure settings which are not configured in the current ".config".

You will most likely be using one of the following three "make menuconfig", "make xconfig", or "make gconfig". The "make menuconfig" option will show you a ncurses-based console UI which will let you configure the kernel using checkboxes, radio-buttons, etc. This is a much more convenient way of configuring your kernel than the methods listed before, however even more convenient and easy to use options exist in "make xconfig" and "make gconfig". These options show a GUI based on Qt, and Gtk respectively which lets you easily browse all the options and read about them as you make your selections.

For each setting that you would like to configure, read the associated help text,



The ncurses-based configuration tool for the kernel